Lab Assignment 1: Requirements Elicitation and Definition Project Implementation, Quality Attribute Analysis

DVA506 Software Architecture

Project Group 4

2026-02-16

Umair Khalid
Balkrishna Panthi
Shaik Omar Farooq
Stina Häggmark
Asli Sümen

Roles

Umair Khalid - Developer
Balkrishna Panthi - Developer
Shaik Omar Farooq - Software Architect
Stina Häggmark - Test Engineer
Asli Sümen - Test Engineer

Functional Requirements (what the system must do)

The focus is on the system's capabilities, features, and specific actions. Examples include user authentication, data processing, generating reports, updating profiles. Functional requirements are tested through pass/fail tests (e.g. does the login button work?)

1. Vehicle lifecycle management

The system should implement a finite state machine for each vehicle. The supported vehicle states are:

- Available
- Reserved
- InUse
- Maintenance
- EmergencyLock
- Relocating (Not implemented due to lack of information)

State transitions
The SmartMoveCentralController (SMCC) must validate all transitions by following these rules:

Transition	**Condition**
Available → Reserved	User requests a reservation
Reserved → InUse	Successful unlock + rental start
InUse → Available	Trip completed
Available → InUse	User unlocks vehicle without reservation
Any → Maintenance	Telemetry reports fault OR battery < 5%
Any → EmergencyLock	Theft detected OR critical overheating
Available → Relocating	Not implemented (due to lack of information)

2. Multi-tenant regulatory logic

The system must support different business logic per city within the same execution flow.

Functional variations per city:
London: Apply congestion charge to trip billing
Milan: Perform helmet presence check before unlocking moped
Rome: Prevent scooters from entering restricted GPS zones
The controller must identify the vehicle's operating city based on GPS data and apply city-specific rules during the rental lifecycle.

3. Real-time telemetry processing

Vehicles send asynchronous telemetry updates containing the following information:

- GPS coordinates
- Battery percentage
- Internal temperature

The SMCC must continuously receive and process streams of this telemetry data in a background process, update the vehicle status in-memory and detect potential rule violations. The interventions needed are as follows:

Temperature > 60°C → Slow vehicle OR emergency rental termination (EmergencyLock)
Battery < 5% during trip → Slow vehicle OR emergency rental termination (Maintenance)
Movement without rental (GPS change while not InUse) → Trigger EmergencyLock state
Entry into restricted zone when in Rome → Enforce restriction (EmergencyLock)

4. Rental transaction handling

The system must support the following different types of rental transactions:

- Vehicle reservation
- Rental start (unlock)
- Rental completion (lock)
- Fare calculation
- Rental termination
- City-specific fees

5. Manual concurrency management

The SMCC must be able to handle concurrent telemetry updates and rental operations by using primitive synchronisation techniques (a such example can be locks) to prevent race conditions when updating vehicle state, processing rentals and writing audit logs.

6. File-based persistence

The system must use local CSV or JSON-files to persist data instead of using a database. This data should include:

- User information
- Vehicle status
- Financial transactions

7. High-integrity audit trail

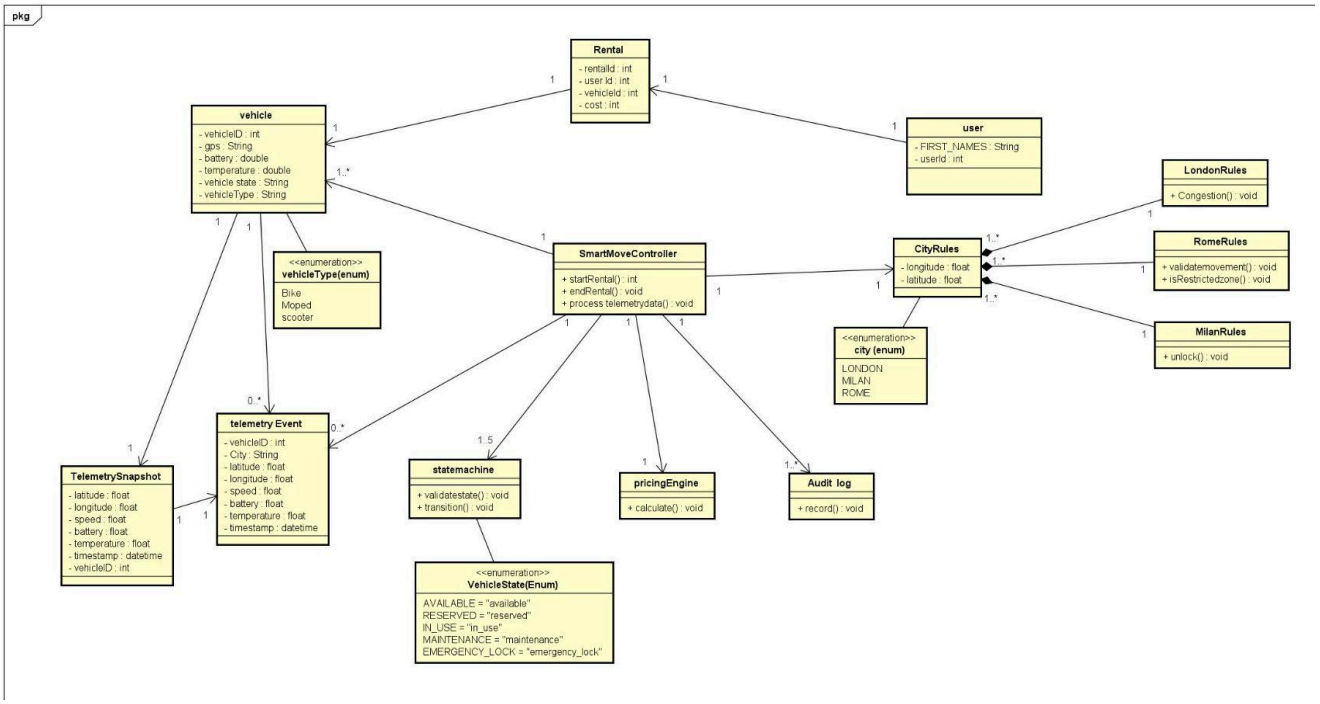
The SMCC must append records to a global audit log every time a vehicle changes state or a payment is processed. Each entry must include a sequential ID and a checksum of the previous entry.

8. Manual rollback mechanism

If a file write fails, the system must revert in-memory changes to ensure that the persisted state from the logs and in-memory state remain consistent.

Non-functional Requirements (how well must the system perform)

The focus is on the system's quality attributes, constraints and characteristics. Examples include performance (response time), security, scalability, usability, reliability, accessibility. Non-functional requirements are measured against benchmarks or metrics (e.g. response time < 2s).

1. **Scalability** -The system must be able to handle at least 10,000 concurrent vehicles, and process continuous telemetry streams from them without degradation.
2. **Real-time responsiveness** - Safety interventions for events like overheating, low battery, or theft, must occur immediately. Telemetry processing must be near real-time.
3. **Data integrity** - State transitions must be atomic, where no partial updates are allowed. In-memory and persisted data must never diverge.
4. **Reliability/fault tolerance** - The system must survive file write failures via a rollback mechanism.
5. **Auditability/compliance** - All critical operations must be traceable through audit logs that are tamper-resistant and verifiable through a checksum.
6. **Persistence constraints** - Must operate without a database, and instead rely on local file storage.
7. **Safety critical behaviour** - The system must prioritise the rider's safety by automated vehicle intervention. The controller shall monitor critical telemetry parameters such as vehicle temperature, battery level, and movement patterns. If predefined safety thresholds are exceeded, the system must immediately trigger automated state transitions to prevent unsafe operation. For example, overheating conditions must result in an EmergencyLock state, while critically low battery levels must force the vehicle into Maintenance mode.

UML Class Diagram